



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial



CÓMPUTO PARALELO

Análisis y diseño de programas paralelos utilizando procesos

Práctica 2

Profesor:

José Alfredo Jiménez Benítez

Presentan:

Cruz Trejo Said Carbot

López Castro Magaly

Romero Rosales Lino Ehecatl

Ugalde Téllez Aarón

Grupo: 6BV1

Ciudad de México, a 28 de septiembre de 2026

Índice general

Índice de figuras	iii
-------------------	-----

Índice de tablas	iv
------------------	----

1. Análisis y diseño de programas paralelos utilizando procesos	1
1.1. Resumen	1
1.2. Introducción	1
1.3. Marco teórico	1
1.3.1. Tipos de datos y representación de matrices en C	2
1.3.2. Formato de almacenamiento del archivo de matriz	2
1.3.3. Procesos en sistemas operativos	3
1.3.4. Manejo de archivos en C	5
1.4. Arquitectura del sistema de procesos múltiples	6
1.4.1. El proceso padre	6
1.4.2. Los procesos hijos	6
1.5. Desarrollo del código	6
1.5.1. Análisis del problema	7
1.5.2. Diseño de la solución	7
1.5.3. Programa 1: creación del archivo de la matriz	8
1.5.4. Programa 2: proceso padre y procesos hijos	8
1.5.5. Compilación con <code>Makefile</code>	10
1.5.6. Validaciones	10
1.6. Evidencias de compilación y ejecución	10
1.6.1. Compilación con <code>make</code>	10
1.6.2. Ejecución del programa 1	10
1.6.3. Ejecución del programa 2	11
1.6.4. Captura de la sesión en la terminal	11
1.7. Conclusiones	12

1.7.1.	Conclusión — Aarón Ugalde Téllez	13
1.7.2.	Conclusión — Said Carbot Cruz Trejo	14
1.7.3.	Conclusión — Magaly López Castro	15
1.7.4.	Conclusión — Lino Ehecatl Romero Rosales	16
1.8.	Anexos	16
1.8.1.	Código de <code>programa1.c</code>	17
1.8.2.	Código de <code>programa2.c</code>	17
1.8.3.	Código del <code>Makefile</code>	20
Referencias		21

Índice de figuras

1.1. Compilación con `make` y ejecución de los programas 1 y 2 en la terminal. 12

Índice de tablas

1.1. Productos esperados por columna de la matriz. 7

Práctica 1. Análisis y diseño de programas paralelos utilizando procesos

1.1 Resumen

En la presente práctica se diseña e implementa un sistema concurrente en lenguaje C utilizando la interfaz para la gestión de procesos. El objetivo principal es paralelizar una tarea de procesamiento de datos: la extracción y multiplicación matemática de las columnas de una matriz de tamaño 3×9 . Mediante el uso de la llamada al sistema `fork()`, un proceso padre orquesta la creación de nueve procesos hijos independientes, asignando a cada uno la carga de trabajo de una columna específica. Para cumplir con las reglas de negocio, la comunicación y persistencia se realiza de forma exclusiva a través de archivos de texto plano, prescindiendo del uso de sockets, memoria compartida o bases de datos. El sistema garantiza la sincronización final mediante la instrucción `wait()`, consolidando los resultados en un archivo de salida de manera ordenada. Este enfoque demuestra los fundamentos de la distribución de cargas computacionales y el control de concurrencia a nivel de sistema operativo.

1.2 Introducción

El diseño de algoritmos concurrentes es una competencia clave en la ingeniería de software y el cómputo de alto rendimiento, áreas fundamentales para disciplinas que manejan cálculos intensivos y grandes volúmenes de datos. En el entorno de sistemas operativos basados en UNIX, la creación de procesos a través de la clonación con `fork()` representa uno de los mecanismos más robustos para lograr la ejecución paralela [1].

A diferencia del manejo de hilos, los procesos generados poseen su propio espacio de direcciones de memoria protegido. Esto proporciona un alto nivel de aislamiento y seguridad durante la ejecución, pero exige estrategias específicas para compartir información y consolidar resultados. En este proyecto se aborda un problema de paralelismo de datos aplicado a una estructura matricial. La estrategia de resolución consiste en delegar la extracción y el procesamiento de cada columna a un proceso hijo distinto, reduciendo teóricamente el tiempo de ejecución en arquitecturas con múltiples núcleos. Para mantener la persistencia y coordinar la salida, se utilizan archivos de texto estándar como medio de intercambio, demostrando cómo orquestar múltiples procesos asegurando la integridad de la información mediante una sincronización adecuada por parte del proceso padre.

1.3 Marco teórico

El guion de la práctica pide investigar tres temas de sistemas operativos y programación en lenguaje C: los tipos de datos, los procesos y los archivos. Las subsecciones siguientes los desarrollan en ese orden, comenzando por la representación de la matriz y el formato del archivo que la almacena.

1.3.1 Tipos de datos y representación de matrices en C

En el lenguaje C, los tipos de datos primitivos determinan la cantidad de memoria asignada a una variable y la forma en que la Unidad Aritmético Lógica (ALU) interpreta sus bits. Para el procesamiento matemático de esta práctica, se utilizan tipos enteros (`int`), los cuales típicamente ocupan 4 bytes en arquitecturas modernas y permiten el almacenamiento exacto de los factores extraídos y los productos calculados.

Una matriz bidimensional en C se define sintácticamente como un arreglo de arreglos (por ejemplo, `int matriz[3][9]`) [2]. Sin embargo, a nivel de arquitectura, la memoria de una computadora es lineal y unidimensional. Por lo tanto, el compilador de C implementa el almacenamiento de matrices utilizando el formato de “orden principal por filas” (*row-major order*). Esto significa que los elementos de la primera fila se almacenan en direcciones de memoria contiguas, seguidos inmediatamente por los elementos de la segunda fila, y así sucesivamente. Para acceder de manera interna al elemento en la posición (i, j) , el lenguaje calcula el desplazamiento utilizando la fórmula:

$$\text{Dirección} = \text{Base} + (i \times \text{Total_Columnas} + j) \times \text{Tamaño_Tipo_Dato}$$

Esta representación es sumamente eficiente para recorrer filas, pero requiere un diseño algorítmico de lectura específico cuando el procesamiento se distribuye y fragmenta por columnas de manera concurrente.

1.3.2 Formato de almacenamiento del archivo de matriz

Dado que la comunicación entre los procesos no emplea sockets, tuberías (*pipes*) ni gestores de bases de datos, la estructura física del archivo de texto plano actúa como el único puente de datos. El programa inicial (o el proceso padre) almacena la matriz de 3×9 en un formato tabular estructurado en código ASCII, delimitado por caracteres de control.

Específicamente, el formato de almacenamiento consiste en tres líneas de texto separadas por secuencias de escape de salto de línea (`\n`). En cada línea, los nueve valores numéricos están delimitados de manera estricta por un espacio en blanco. Visualmente y a nivel de bytes, el archivo se compone de la siguiente manera:

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 1 & 3 & 5 & 7 & 9 & 11 & 13 & 15 & 17 \\ 2 & 4 & 6 & 8 & 10 & 12 & 14 & 16 & 18 \end{bmatrix}$$

Este formato de almacenamiento secuencial permite que cada uno de los 9 procesos hijos abra el archivo en modo de solo lectura (`"r"`). Utilizando funciones estándar de la biblioteca `<stdio.h>`, como `fscanf()` o `fgets()` en combinación con `strtok()`, cada hijo puede posicionarse en el flujo del archivo, saltar los valores irrelevantes, y extraer iterativamente solo el n -ésimo token de cada fila que corresponda exclusivamente a su columna asignada. Esta simplicidad estructural evita la necesidad de implementar librerías de parseo complejas, manteniendo la lectura paralela segura y concurrente.

1.3.3 Procesos en sistemas operativos

En el ámbito de los sistemas operativos (SO), un proceso se define fundamentalmente como un programa en ejecución. Sin embargo, es más que simplemente el código del programa (a menudo denominado sección de texto); incluye la actividad actual representada por el valor del contador de programa (*Program Counter*) y el contenido de los registros del procesador. Además, un proceso generalmente incluye la pila (*stack*) del proceso, que contiene datos temporales como parámetros de funciones, direcciones de retorno y variables locales, y una sección de datos que contiene variables globales [3].

Identificadores de procesos (PID y PPID)

Para gestionar la multiplicidad de procesos que coexisten en un sistema, el sistema operativo debe poder identificarlos de manera unívoca.

- **PID (*Process Identifier*):** es un número entero único asignado por el sistema operativo en el momento de la creación del proceso. El SO utiliza el PID para rastrear el proceso y gestionar la asignación de recursos, permisos y la planificación (*scheduling*) [4].
- **PPID (*Parent Process Identifier*):** los procesos en la mayoría de los sistemas operativos modernos se crean jerárquicamente. El proceso que crea un nuevo proceso se denomina proceso padre. El PPID es el identificador del proceso padre. Esta estructura de árbol es esencial para la gestión de recursos y la herencia de propiedades, así como para la propagación de señales en sistemas similares a UNIX [5].

Llamadas al sistema para el control de procesos

La interfaz entre los procesos en el espacio de usuario y el núcleo (*kernel*) del sistema operativo se realiza a través de llamadas al sistema (*system calls*). Para la gestión del ciclo de vida de los procesos en sistemas compatibles con POSIX (como Linux y macOS), destacan cuatro funciones fundamentales [4]:

- `fork()`: esta llamada al sistema se utiliza para crear un nuevo proceso. El proceso creado (hijo) es una copia casi exacta del proceso creador (padre). Ambos procesos continúan su ejecución en la instrucción que sigue al `fork()`. La diferencia clave radica en el valor de retorno de la función: devuelve el PID del hijo al proceso padre, y devuelve 0 al proceso hijo, permitiendo que ambos ejecuten bloques lógicos distintos.
- `exit()`: utilizada para terminar la ejecución de un proceso de manera normal. Cuando un proceso llama a `exit()`, el SO libera los recursos asignados a él (memoria, archivos abiertos, etc.). El proceso envía un código de estado (*status*) de terminación a su proceso padre.
- `wait()`: un proceso padre utiliza `wait()` para suspender su propia ejecución hasta que uno de sus procesos hijos termine. Si el hijo ya ha terminado (convirtiéndose temporalmente en un proceso “zombie”), `wait()` retorna inmediatamente con el estado de terminación del hijo, permitiendo al sistema limpiar la entrada del hijo en la tabla de procesos [6].

- `waitpid()`: proporciona una versión extendida de `wait()`. Permite al proceso padre especificar exactamente por qué proceso hijo desea esperar, utilizando su PID. Además, ofrece opciones para no bloquear la ejecución (espera no bloqueante) si el hijo aún no ha cambiado de estado.

Ciclo de vida de los procesos

Desde su creación hasta su terminación, un proceso transita por diversos estados. Aunque los modelos pueden variar según el SO, el modelo genérico de cinco estados es el más aceptado [3]:

1. **Nuevo (*New*)**: el proceso está siendo creado por el sistema operativo, pero aún no ha sido admitido en el grupo de procesos ejecutables.
2. **Listo (*Ready*)**: el proceso está en la memoria principal y está disponible para su ejecución, esperando que el planificador (*scheduler*) del SO le asigne tiempo de CPU.
3. **En ejecución (*Running*)**: las instrucciones del proceso están siendo ejecutadas actualmente por la CPU.
4. **Bloqueado o en espera (*Waiting/Blocked*)**: el proceso no puede continuar ejecutándose porque está esperando que ocurra un evento externo (como la finalización de una operación de entrada/salida o la recepción de una señal).
5. **Terminado (*Terminated*)**: el proceso ha finalizado su ejecución (ya sea normalmente vía `exit()` o por un error) y el sistema operativo está reclamando sus recursos.

Concurrencia frente a paralelismo a nivel de sistema operativo

Es fundamental distinguir entre concurrencia y paralelismo en la teoría de sistemas operativos, aunque a menudo se utilizan indistintamente de manera errónea [5].

- **Concurrencia**: es la capacidad del SO para manejar múltiples procesos al mismo tiempo mediante la técnica de multiplexación en el tiempo. En un sistema de un solo procesador (un solo núcleo), la CPU intercala la ejecución de varios procesos, ejecutando un proceso por un breve lapso (*quantum* de tiempo) y luego cambiando a otro (cambio de contexto). Esto crea la *ilusión* de ejecución simultánea, aunque en realidad las instrucciones se ejecutan secuencialmente.
- **Paralelismo**: implica la ejecución física simultánea de múltiples procesos o hilos de un proceso. Para que exista paralelismo real a nivel de hardware, el sistema debe poseer múltiples unidades de procesamiento (procesadores multinúcleo o sistemas multiprocesador). En este escenario, dos o más procesos ejecutan sus instrucciones en el mismo instante temporal en núcleos diferentes [6].

Sincronización de procesos

En entornos concurrentes o paralelos, los procesos a menudo necesitan compartir recursos (memoria, archivos, dispositivos). Cuando varios procesos intentan acceder y modificar datos compartidos de manera no coordinada, se pueden producir inconsistencias, un fenómeno conocido como *condición de carrera* (*race condition*) [3].

La **sincronización de procesos** es el conjunto de mecanismos proporcionados por el SO para controlar el orden de ejecución de los procesos y garantizar la integridad de los datos compartidos. El objetivo principal es resolver el problema de la *sección crítica*, que es la parte del código de un proceso donde se accede al recurso compartido.

Para lograr la sincronización, los SO implementan herramientas como:

- **Mutua exclusión (*mutex*):** bloqueos que garantizan que solo un proceso a la vez pueda entrar en su sección crítica.
- **Semáforos:** variables enteras que, mediante operaciones atómicas especializadas (`wait()` y `signal()`), también conocidas como *P* y *V*), controlan el acceso a recursos que pueden tener un número limitado de instancias permitidas.

Sin una adecuada sincronización, los sistemas concurrentes serían inestables, llevando a bloqueos mutuos (*deadlocks*) o corrupción de información [5].

1.3.4 Manejo de archivos en C

El lenguaje C proporciona un conjunto de funciones dentro de su biblioteca estándar para la manipulación de archivos, permitiendo la persistencia de datos mediante flujos [1]. Debido a las restricciones de diseño que impiden el uso de gestores de bases de datos, el almacenamiento y recuperación de la información del sistema dependerá exclusivamente de las siguientes operaciones de entrada y salida a nivel de archivo:

- **`fopen()`**: esta función abre un archivo y lo asocia a un flujo. Devuelve un puntero de tipo `FILE` que funciona como identificador para todas las operaciones subsecuentes sobre dicho archivo. En caso de que la apertura falle, la función devuelve `NULL`.
- **Modos de apertura:** el segundo parámetro de `fopen()` define el tipo de acceso permitido:
 - `"r"` (*read*): abre el archivo en modo de solo lectura. Requiere que el archivo ya exista.
 - `"w"` (*write*): abre el archivo en modo de escritura. Si el archivo existe, sobrescribe su contenido truncándolo a cero. Si no existe, lo crea.
 - `"a"` (*append*): abre el archivo para añadir datos al final del mismo, conservando el contenido previo.
- **`fprintf()`**: opera de manera idéntica a `printf()`, con la diferencia de que escribe la salida formateada directamente en el flujo del archivo especificado.
- **`fscanf()`**: diseñada para leer datos formateados desde un flujo de archivo. Resulta esencial para extraer la información previamente almacenada y cargarla en memoria.
- **`fclose()`**: cierra el flujo asociado al archivo, vaciando (*flushing*) el búfer de salida y liberando los recursos del sistema operativo. Es crítico invocarla para evitar la corrupción de datos.

1.4 Arquitectura del sistema de procesos múltiples

El diseño del sistema se fundamenta en un modelo de concurrencia basado en la creación de múltiples procesos utilizando el estándar POSIX del lenguaje C [1]. Dado que las reglas de negocio prohíben explícitamente el uso de *sockets* para la comunicación entre procesos y de bases de datos para el almacenamiento, la arquitectura se sostiene sobre la bifurcación de procesos en el sistema operativo y la concurrencia controlada sobre el sistema de archivos local.

La arquitectura se divide en dos componentes técnicos principales: el proceso padre y los procesos hijos.

1.4.1 El proceso padre

El proceso principal actúa como el controlador del sistema. Sus responsabilidades arquitectónicas son:

- **Validación:** verifica que exista el archivo de entrada y comprueba el resultado de cada operación que puede fallar (apertura de archivos, lectura, escritura y creación de procesos). El sistema no captura datos del usuario por consola: la matriz y los nombres de archivo son constantes del programa.
- **Bifurcación (`fork()`):** utiliza la llamada al sistema `fork()` para clonar su espacio de direcciones y generar los procesos hijos [1].
- **Sincronización (`wait()`):** para evitar la proliferación de procesos *zombies*, el padre suspende su ejecución mediante la función `wait()` hasta que todos los procesos hijos han reportado su terminación.

1.4.2 Los procesos hijos

Cada proceso hijo asume un ciclo de vida independiente, con una copia propia del espacio de direcciones del padre.

- **Ejecución paralela:** al evaluar el valor de retorno 0 tras la llamada a `fork()`, el flujo de control del hijo se activa para ejecutar su lógica de negocio de manera concurrente.
- **Gestión de entradas y salidas:** el intercambio indirecto de información y la persistencia del estado se realizan abriendo los archivos compartidos mediante `fopen()` y registrando los resultados con `fprintf()`.
- **Finalización (`exit()`):** al concluir su trabajo, el proceso hijo invoca `exit()` para liberar su memoria y notificar su término exitoso mediante la señal `SIGCHLD`.

1.5 Desarrollo del código

Esta sección describe el análisis, el diseño y la implementación de los programas escritos en lenguaje C que resuelven la práctica, a partir de la arquitectura presentada en la sección 1.4. El código completo se encuentra en los anexos (sección 1.8).

1.5.1 Análisis del problema

El enunciado pide un sistema de procesos que lea un archivo con una matriz de tres filas y nueve columnas, calcule el producto de los elementos de cada columna y guarde los resultados en otro archivo. Los datos de la matriz y su formato en el archivo se presentaron en la sección 1.3.2.

El cálculo de una columna no necesita ningún dato de las demás, por lo que el problema es *naturalmente paralelizable*: se puede asignar una columna a cada proceso y todos pueden trabajar al mismo tiempo sin comunicarse durante el cálculo. Con nueve columnas se requieren nueve procesos hijos. La tabla 1.1 muestra el resultado esperado de cada columna, calculado a mano, que sirve para verificar la salida de los programas.

Tabla 1.1. Productos esperados por columna de la matriz.

Columna	Elementos	Producto
1	$1 \cdot 1 \cdot 2$	2
2	$2 \cdot 3 \cdot 4$	24
3	$3 \cdot 5 \cdot 6$	90
4	$4 \cdot 7 \cdot 8$	224
5	$5 \cdot 9 \cdot 10$	450
6	$6 \cdot 11 \cdot 12$	792
7	$7 \cdot 13 \cdot 14$	1274
8	$8 \cdot 15 \cdot 16$	1920
9	$9 \cdot 17 \cdot 18$	2754

1.5.2 Diseño de la solución

La solución se dividió en dos programas independientes, que se compilan con un mismo `Makefile`:

- **programa1**: crea el archivo `matriz.txt` y guarda en él la matriz, una fila por línea y con los elementos separados por espacios.
- **programa2**: es el proceso padre. Crea el archivo de resultados, genera un proceso hijo por cada columna, espera a que todos terminen y muestra el contenido del archivo de resultados.

El flujo de `programa2` sigue el orden que pide el enunciado:

1. El padre verifica que exista `matriz.txt`; si no existe, avisa que debe ejecutarse primero el programa 1 y termina.
2. El padre crea (o vacía) el archivo `resultados.txt`.
3. El padre crea nueve procesos hijos con `fork()`, uno por columna, e informa a cada uno qué columna le toca.
4. Cada hijo abre `matriz.txt`, extrae los datos, multiplica los elementos de su columna y agrega el resultado a `resultados.txt`.

5. El padre espera con `wait()` a que terminen los nueve hijos y revisa el estado de salida de cada uno.
6. El padre lee `resultados.txt` y muestra su contenido en pantalla.

Como la práctica prohíbe el uso de *sockets*, no hay comunicación directa entre procesos. La información fluye únicamente por dos vías: el número de columna, que el hijo recibe al momento de su creación, y los archivos, que sirven como medio compartido para los datos de entrada y los resultados.

1.5.3 Programa 1: creación del archivo de la matriz

El programa 1 (código 1.4) declara la matriz como un arreglo bidimensional de enteros de 3×9 , inicializado con los valores del enunciado, y las dimensiones se definen con constantes (`FILAS` y `COLUMNAS`) para evitar números fijos repartidos por el código. Abre `matriz.txt` en modo escritura ("`w`"), con lo que el archivo se crea si no existe y se sobrescribe si ya existía, y escribe cada elemento con `fprintf()`, separando las columnas con un espacio y las filas con un salto de línea. Al no dejar un espacio al final de cada fila, el archivo queda con el mismo formato que la matriz impresa en el enunciado.

Se validan los dos puntos donde es más probable que haya un error: la apertura del archivo (con `perror()` para mostrar la causa) y la escritura de cada elemento con `fprintf()`, cuyo valor de retorno negativo indica un fallo. Ante cualquier error el programa cierra el archivo, si ya estaba abierto, y termina con `EXIT_FAILURE`. Si todo salió bien, muestra un mensaje de confirmación.

1.5.4 Programa 2: proceso padre y procesos hijos

Proceso padre

El padre (función `main` del código 1.5) primero comprueba que `matriz.txt` exista, abriéndolo en modo lectura, y a continuación abre `resultados.txt` en modo "`w`" y lo cierra de inmediato. Este paso deja el archivo vacío antes de que existan los hijos, de forma que los resultados de una ejecución anterior no se mezclen con los actuales y que ningún hijo tenga que decidir si debe borrar el archivo, lo cual podría destruir lo que ya escribió otro hijo.

Después, un ciclo crea un hijo por columna:

```
1  for (int col = 0; col < COLUMNAS; col++) {
2      pid_t pid = fork();
3
4      if (pid < 0) {
5          perror("[PADRE] Error al ejecutar fork()");
6          return EXIT_FAILURE;
7      } else if (pid == 0) {
8          // Proceso Hijo
9          tarea_hijo(col);
10     }
11 }
```

La llamada `fork()` devuelve un valor negativo si falló, cero en el proceso hijo y el PID del hijo en el padre. Cuando el valor es cero, el hijo ejecuta `tarea_hijo(col)`, que termina con `exit()` y por lo tanto nunca regresa al ciclo. Esto es importante: si los hijos continuaran ejecutando el ciclo, cada uno crearía a su vez más procesos. Así, es el padre el único que completa las nueve iteraciones.

La forma de informar a cada hijo qué columna le toca aprovechar el propio mecanismo de `fork()`: el hijo nace con una copia de la memoria del padre, incluida la variable `col` con el valor que tenía en el momento de la llamada, y ese valor se entrega como argumento a `tarea_hijo()`.

Justo antes del ciclo, el padre imprime el mensaje «Creando 9 procesos hijos» y llama a `fflush(stdout)`. Esta llamada es necesaria porque, cuando la salida estándar no es una terminal (por ejemplo, si se redirige a un archivo o a otro comando), el sistema la mantiene en un búfer y no la imprime hasta que se llena o el programa termina. Como `fork()` copia también ese búfer, cada uno de los nueve hijos heredaría el mensaje aún sin imprimir y lo volcaría al ejecutar `exit()`, con lo que el texto aparecería diez veces en lugar de una. Vaciar el búfer antes de crear a los hijos evita esa repetición.

Una vez creados todos los hijos, otro ciclo llama nueve veces a `wait()`, que bloquea al padre hasta que termina cualquiera de sus hijos. Con el estado devuelto se verifica, con `WIFEXITED` y `WEXITSTATUS`, que el hijo haya terminado normalmente y con código cero; si no fue así, se muestra una advertencia con el PID del hijo afectado. Finalmente, el padre abre `resultados.txt` en modo lectura, imprime su contenido línea por línea con `fgets()` y termina.

Proceso hijo

La función `tarea_hijo()` realiza el trabajo de cada hijo en tres pasos. Primero abre `matriz.txt` en modo lectura y carga la matriz completa con `fscanf()`, validando que cada lectura devuelva exactamente un dato. Aunque cada hijo sólo necesita una columna, el archivo está escrito por filas y el formato de texto obliga a recorrerlo completo para llegar a los elementos de una columna dada. Segundo, multiplica los elementos de esa columna:

```
1 long long producto = 1;
2 for (int i = 0; i < FILAS; i++) {
3     producto *= matriz[i][col];
4 }
```

El producto se acumula en una variable `long long`, que tiene un rango mucho mayor que un `int` y evita desbordamientos si los valores o el tamaño de la matriz crecen. Tercero, abre `resultados.txt` en modo agregar ("a") y escribe una línea con el formato `Columna N: producto`. El modo "a" hace que cada escritura se agregue al final del archivo, lo que permite que los nueve hijos escriban en el mismo archivo sin sobrescribirse entre sí. Al terminar, el hijo cierra el archivo y sale con `EXIT_SUCCESS`, o con `EXIT_FAILURE` si algún paso falló, información que el padre recupera con `wait()`.

Como todos los hijos se ejecutan de forma concurrente, el orden en que escriben sus líneas depende del planificador del sistema operativo y puede variar entre ejecuciones. Por esa razón cada línea lleva el número de columna, de modo que el resultado se interpreta correctamente sin importar el orden de llegada.

1.5.5 Compilación con Makefile

La compilación se automatiza con un Makefile (código 1.6) que define el compilador (`gcc`) y las banderas `-Wall -Wextra -std=c11`, con las que se activan las advertencias más importantes para que el código compile sin errores ni avisos, como pide la práctica. El objetivo `all`, que es el predeterminado, construye los dos ejecutables, cada uno con su propia regla de dependencia de su archivo fuente. El objetivo `clean` borra los ejecutables y los archivos de datos generados, y ambos objetivos se declaran como `.PHONY` para que `make` los ejecute aunque exista un archivo con el mismo nombre.

Para usar el sistema se compila con `make` y se ejecutan en orden `./programa1` y `./programa2`, porque el segundo depende del archivo que crea el primero.

1.5.6 Validaciones

Los programas no piden datos al usuario, ya que la matriz está fija en el código y los nombres de archivo son constantes. Por ello la validación se concentró en las operaciones que pueden fallar durante la ejecución: apertura de archivos (`fopen()`), lectura (`fscanf()`), escritura (`fprintf()`), creación de procesos (`fork()`) y estado de terminación de los hijos (`wait()`). Además, el programa 2 verifica al inicio que exista el archivo de entrada y, en caso contrario, indica al usuario cómo resolver el problema.

1.6 Evidencias de compilación y ejecución

Las evidencias de esta sección se obtuvieron en un equipo Apple Silicon (arquitectura `arm64`) con macOS, donde el comando `gcc` corresponde al compilador Apple clang 21 de las herramientas de línea de comandos del sistema. Los archivos fuente son los mismos que se listan en los anexos (sección 1.8).

1.6.1 Compilación con make

Desde el directorio de la práctica se ejecuta `make`. El Makefile invoca a `gcc` con las banderas `-Wall -Wextra -std=c11` para cada programa. Como muestra el código 1.1, la compilación termina sin errores ni advertencias: no aparece ningún mensaje del compilador más allá de los dos comandos ejecutados, y `make` devuelve código de salida cero.

```
$ make
gcc -Wall -Wextra -std=c11 -o programa1 programa1.c
gcc -Wall -Wextra -std=c11 -o programa2 programa2.c
```

Código 1.1. Compilación de los dos programas con `make`.

1.6.2 Ejecución del programa 1

El programa 1 crea el archivo `matriz.txt`. Al mostrar su contenido con `cat` (código 1.2) se comprueba que la matriz quedó guardada como se describió en la sección 1.3.2: tres líneas con nueve valores separados por espacios.

```
$ ./programa1
[PROGRAMA 1] El archivo 'matriz.txt' se ha generado exitosamente.
$ cat matriz.txt
1 2 3 4 5 6 7 8 9
1 3 5 7 9 11 13 15 17
2 4 6 8 10 12 14 16 18
```

Código 1.2. Ejecución del programa 1 y contenido de `matriz.txt`.

1.6.3 Ejecución del programa 2

El programa 2 es el proceso padre: crea los nueve procesos hijos, espera a que terminen y muestra el contenido de `resultados.txt` (código 1.3). Los nueve productos coinciden con los valores esperados de la tabla 1.1. En esta ejecución las líneas aparecieron en el orden de las columnas, pero, como se explicó en la sección 1.5.4, ese orden no está garantizado, porque depende de cuál hijo termine primero; por eso cada línea indica el número de columna.

```
$ ./programa2
[PADRE] Creando 9 procesos hijos para procesar las columnas en paralelo...

[PADRE] Todos los hijos han terminado. Resultados contenidos en
      'resultados.txt':
-----
Columna 1: 2
Columna 2: 24
Columna 3: 90
Columna 4: 224
Columna 5: 450
Columna 6: 792
Columna 7: 1274
Columna 8: 1920
Columna 9: 2754
-----
```

Código 1.3. Ejecución del programa 2 (proceso padre y nueve procesos hijos).

1.6.4 Captura de la sesión en la terminal

La figura 1.1 muestra la sesión completa en la terminal: el cambio al directorio de la práctica, la compilación con `make` y la ejecución de los programas 1 y 2. En esta captura `make` sólo recompiló `programa2`, porque `programa1` ya estaba compilado y actualizado; en ambos casos la compilación termina sin errores ni advertencias. La salida del programa 2 muestra los nueve productos por columna, que coinciden con los valores esperados de la tabla 1.1.


```
● → Proyecto-Paralelo git:(master) x cd practicas/prac02
● → prac02 git:(master) x make
gcc -Wall -Wextra -std=c11 -o programa2 programa2.c
● → prac02 git:(master) x ./programa1
[PROGRAMA 1] El archivo 'matriz.txt' se ha generado exitosamente.
● → prac02 git:(master) x ./programa2
[PADRE] Creando 9 procesos hijos para procesar las columnas en paralelo...

[PADRE] Todos los hijos han terminado. Resultados contenidos en 'resultados.txt':
-----
Columna 1: 2
Columna 2: 24
Columna 3: 90
Columna 4: 224
Columna 5: 450
Columna 6: 792
Columna 7: 1274
Columna 8: 1920
Columna 9: 2754
-----
```

Figura 1.1. Compilación con make y ejecución de los programas 1 y 2 en la terminal.

1.7 Conclusiones

A continuación cada integrante del equipo comparte su experiencia individual con el desarrollo de esta práctica.

1.7.1 Conclusión — Aarón Ugalde Téllez

En esta práctica pude ver de forma concreta lo que significa dividir un problema entre varios procesos. La matriz de tres filas y nueve columnas es pequeña, pero su estructura se presta bien al paralelismo: el producto de los elementos de una columna no depende de ninguna otra columna, así que cada una puede calcularse por separado sin que los procesos tengan que coordinarse entre sí durante el cálculo. Nuestro diseño quedó en dos programas. El primero genera el archivo `matriz.txt` con los datos, y el segundo actúa como proceso padre: crea el archivo de resultados, lanza nueve procesos hijos con `fork()`, espera a que todos terminen con `wait()` y muestra al final el contenido de `resultados.txt`.

Uno de los puntos que más me hizo pensar fue cómo informar a cada hijo qué columna le corresponde sin usar *sockets*, que la práctica prohíbe. La solución resultó más sencilla de lo que esperaba: al ejecutar `fork()`, el hijo hereda una copia de la memoria del padre, incluida la variable del ciclo que indica la columna en turno, de modo que basta con pasarla como argumento a la función que ejecuta el hijo. Esto me ayudó a entender que un proceso hijo no comparte memoria con su padre, sino que recibe una copia independiente. Por eso, lo que un hijo modifique en sus variables no le llega al padre, y por eso los resultados tienen que viajar por un archivo. Ahí aparece, además, la razón de usar el modo de apertura "a" (*append*) en el archivo de resultados: como los nueve hijos escriben casi al mismo tiempo, cada uno debe agregar su línea al final sin borrar lo que ya escribieron los demás. También entendí por qué el padre crea y vacía `resultados.txt` antes de generar a los hijos: si lo hicieran ellos, uno podría borrar el trabajo de otro.

Otra consecuencia del paralelismo es que el orden en que terminan los hijos lo decide el sistema operativo y no el programa, así que las líneas de `resultados.txt` no tienen garantizado el orden de las columnas. Por eso cada línea incluye el número de columna, de modo que el resultado sigue siendo interpretable aunque llegue desordenado. Además, como no hay datos capturados por el usuario, la validación se concentró en revisar el valor de retorno de cada operación que puede fallar: la apertura de archivos, la lectura con `fscanf()`, la escritura con `fprintf()` y la propia llamada a `fork()`. Para el padre, verificar el estado de salida de cada hijo fue importante, porque un hijo puede fallar sin que el padre lo note si no revisa lo que devuelve `wait()`. Elegí `long long` para los productos por prudencia, ya que multiplicar valores enteros puede desbordar un `int` con mucha facilidad en cuanto la matriz crezca.

Finalmente, el uso de un `Makefile` con las banderas `-Wall` y `-Wextra` me obligó a escribir código sin advertencias, lo cual es un buen hábito para las prácticas que siguen. Esta práctica me deja una base clara para los temas posteriores de la materia: sé cómo crear procesos, cómo esperar su terminación y cómo repartirles trabajo, y también sé que su costo (crear un proceso completo por cada columna) sólo se justifica cuando el trabajo de cada uno es considerable, algo que será útil al comparar procesos con hilos más adelante.

1.7.2 Conclusión — Said Carbot Cruz Trejo

El desarrollo de la Práctica 2 representó un reto enorme y me enseñó bastante sobre cómo los sistemas operativos controlaban el hardware real. Desde mi rol enfocado en la investigación teórica, analicé a fondo los conceptos detrás del manejo de memoria y la concurrencia. Al observar cómo el equipo usó la función `fork()` para dividir las tareas, vi la teoría del paralelismo convertida en algo vivo y funcional. Estudié cómo el programa partió el trabajo de la matriz original de tres filas por nueve columnas y le asignó una columna específica a un proceso hijo distinto. Analizar esto me hizo cambiar mi forma de pensar sobre la ejecución de código. Mandar la instrucción para crear procesos resultó fácil, pero controlarlos requirió un diseño lógico súper riguroso para que los programas no chocaran al buscar la información ni se quedaran colgados en la memoria RAM.

El mayor obstáculo llegó cuando el equipo cumplió la regla de no usar herramientas modernas de comunicación, como bases de datos o sockets. Al depender solamente de leer y escribir en archivos de texto plano, noté lo vital que era dominar las entradas y salidas básicas del lenguaje C. Revisé paso a paso cómo cada proceso hijo abría el documento inicial, leía línea por línea, saltaba los números que no le servían y extraía solo sus tres valores asignados. Después, vi cómo el código multiplicó eso y guardó el resultado utilizando el modo de agregar texto (“a”). Esta simple letra permitió que los nueve programas escribieran en el mismo lugar sin pisarse los talones ni borrar el trabajo de sus compañeros. Además, la función `wait()` me recordó mucho a la sincronización estricta que requieren los sistemas distribuidos. El proceso padre actuó como un coordinador absoluto que pausó su propio reloj y esperó a que todas las tareas terminaran antes de juntar las piezas y mostrar el final en la pantalla.

Más allá del código puro, esta práctica conectó directo con los problemas reales que enfrento en la computación actual. Segmentar y multiplicar partes de una matriz pareció un simple ejercicio de salón, pero en realidad, descubrí que esta es la base matemática de lo que hago todo el tiempo. Por ejemplo, en la simulación de circuitos cuánticos usando Qiskit o PennyLane, aplicar compuertas cuánticas a un registro de cúbits Transmon se reduce, en el fondo, a multiplicaciones de matrices gigantescas contra vectores de estado. Cuando trabajé en la optimización de algoritmos híbridos como el VQE (*Variational Quantum Eigensolver*) para simulaciones de química o al estructurar generadores de números aleatorios para el modelado de mercados financieros, noté que la única forma de procesar esos espacios de estado masivos era paralelizando el trabajo clásico exactamente de esta misma forma. Entender cómo el sistema operativo reparte el procesador entre tareas simultáneas me ayudó a dimensionar el verdadero cuello de botella de los simuladores. En Inteligencia Artificial, solemos usar *frameworks* que esconden toda esta talacha matemática, pero analizar este sistema en C me permitió ver el costo computacional real.

Finalmente, evalué cómo el equipo usó un archivo `Makefile` para darle instrucciones súper estrictas al compilador y cómo limpiaron absolutamente todas las advertencias de la terminal. Esto le dio una calidad técnica excelente a nuestra entrega y demostró la importancia de las buenas prácticas. En conclusión, esta práctica mejoró muchísimo mi visión sobre la programación pura y transformó conceptos teóricos abstractos en cosas que pude medir. Aprendí cómo la teoría de estructuras de datos y el manejo lineal de la memoria respaldan cada línea de código cuando buscamos velocidad y seguridad. Todo este esfuerzo construyó una base estructural valiosísima que sin duda me servirá para diseñar mejores flujos de trabajo paralelos, tanto en arquitecturas de IA clásicas como en mis futuros proyectos de computación cuántica distribuida.

1.7.3 Conclusión — Magaly López Castro

El desarrollo de esta práctica representó una oportunidad invaluable para aterrizar los conceptos teóricos de la gestión de procesos mediante su implementación directa en el lenguaje C. Ejecutar y depurar este código en un entorno de terminal Linux me permitió visualizar claramente la estructura jerárquica que forma el sistema operativo al gestionar la ejecución concurrente. Al diseñar un programa donde un proceso padre actúa como orquestador delegando tareas específicas a múltiples procesos hijos, la teoría sobre multiplexación y distribución de carga de trabajo se volvió completamente tangible.

El núcleo del ejercicio radicó en la correcta utilización de la llamada al sistema `fork()`. A través de ella, fue posible generar copias del proceso original, donde el gran reto algorítmico consistió en utilizar el control de flujo para que el padre informara a cada hijo exactamente qué columna de la matriz debía procesar. Esto demostró que la concurrencia no solo trata de crear múltiples procesos, sino de dotarlos de contexto e identidad lógica para evitar el procesamiento redundante.

Por otro lado, la interacción con los tipos de datos y los archivos añadió una capa vital de complejidad. Al tener múltiples procesos hijos abriendo un mismo archivo de manera concurrente para extraer información, fue necesario garantizar que cada uno leyera únicamente los datos correspondientes a su columna asignada antes de realizar la multiplicación de los elementos. Posteriormente, la escritura de estos resultados parciales en un nuevo archivo de salida puso a prueba el manejo de los descriptores de archivo en C, evidenciando los riesgos de las condiciones de carrera si no se gestionan adecuadamente las operaciones de entrada/salida.

Finalmente, comprendí el rol crítico de la llamada `wait()` en la sincronización. Si el proceso padre no hubiera bloqueado su ejecución a la espera de la terminación de todos sus hijos, se habría intentado mostrar un resultado final basado en un archivo de texto incompleto, generando además procesos huérfanos en el sistema. Dominar estas primitivas básicas de POSIX no solo resuelve el problema matricial planteado, sino que sienta unas bases metodológicas muy sólidas para comprender modelos de cómputo mucho más robustos, facilitando la transición futura hacia el estudio de arquitecturas de paralelismo real y supercómputo, como el uso de *frameworks* al estilo de OpenMP o MPI.

1.7.4 Conclusión — Lino Ehecatl Romero Rosales

El desarrollo de esta práctica resultó bastante interesante porque nos permitió aplicar una forma distinta de pensar al momento de diseñar el sistema. En la carrera ya estamos muy acostumbrados a hacer código y resolver problemas lógicos, pero pasar de un compilar un programa a utilizar procesos paralelos me hizo comprender un enfoque mucho más profundo, más cercano a cómo administra los recursos el sistema operativo. Usar la función `fork` para ramificar el programa es algo que requiere atención, no porque la programación sea complicada, sino porque hay que tener cuidado con la forma en que se maneja la memoria y el ciclo de vida de cada proceso, para asegurarnos de que el sistema termine de forma limpia y sin dejar procesos zombies.

La dinámica que aplicamos para resolver el problema me pareció un muy buen ejemplo de cómo dividir la carga de trabajo. Partimos de una matriz de 3 filas y 9 columnas, donde el proceso del padre se encargó de crear un hijo por cada una de las columnas. Cada proceso hijo tenía la tarea independientemente de leer los datos de su propia columna y multiplicar esos valores. Aunque hacer esto de forma lineal sería instantáneo, implementar esta arquitectura donde varios procesos trabajan al mismo tiempo sobre la misma información es la base para escalar sistemas. Especialmente en el área de inteligencia artificial y análisis de datos, saber cómo repartir el trabajo para procesar información de manera concurrente es algo fundamental.

Un requerimiento clave de esta entrega fue la regla de gestionar la información sin bases de datos y sin usar sockets para la comunicación. Tuvimos que lograr que los procesos trabajaran coordinados utilizando únicamente el disco duro a través de archivos. Usamos las funciones clásicas de C como `fopen`, `fscanf` y `fprintf` para que cada hijo abriera el archivo original, extrajera su columna y escribiera el resultado final en el archivo de salida. Ya tenemos bastante experiencia manejando archivos, pero coordinar de forma concurrente las entradas y salidas de 9 procesos sin que los datos se corrompan o se sobrescriban te obliga a tener un control mucho más estricto del estado del sistema.

Toda esta arquitectura no funcionaría correctamente sin una buena sincronización. Para eso implementamos la función `wait`, que permite al proceso padre quedarse en espera hasta que todos sus hijos terminen de multiplicar y guardar sus respectivos resultados. Si el padre no coordina esto y se adelanta, el programa terminaría su ejecución dejando el archivo de salida a medias. Esto demuestra que la eficiencia en paralelo no se trata nada más de lanzar tareas al mismo tiempo, sino de saber orquestarlas.

Para cerrar, considero que esta práctica fue una excelente manera de fortalecer lo que ya sabemos hacer al programar, pero llevándolo a un nivel de optimización superior. Ver en la práctica cómo los procesos conviven, comparten archivos y se sincronizan mediante señales del sistema, me da mucha más confianza para seguir construyendo arquitecturas de software robustas. Al final, el objetivo ya no es solo que el algoritmo funcione y dé el resultado correcto, sino que lo haga aprovechando el hardware al máximo.

1.8 Anexos

En este apartado se incluye el código fuente completo generado para la práctica: los dos programas en lenguaje C y el `Makefile` con el que se compilan.

1.8.1 Código de programa1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define FILAS 3
5 #define COLUMNAS 9
6 #define ARCHIVO_MATRIZ "matriz.txt"
7
8 int main(void) {
9     int matriz[FILAS][COLUMNAS] = {
10         {1, 2, 3, 4, 5, 6, 7, 8, 9},
11         {1, 3, 5, 7, 9, 11, 13, 15, 17},
12         {2, 4, 6, 8, 10, 12, 14, 16, 18}
13     };
14
15     FILE *archivo = fopen(ARCHIVO_MATRIZ, "w");
16     if (archivo == NULL) {
17         perror("Error al crear el archivo de la matriz");
18         return EXIT_FAILURE;
19     }
20
21     for (int i = 0; i < FILAS; i++) {
22         for (int j = 0; j < COLUMNAS; j++) {
23             if (fprintf(archivo, "%d%s", matriz[i][j], (j == COLUMNAS - 1) ? ""
24                 : " ") < 0) {
25                 fprintf(stderr, "Error al escribir datos en %s\n",
26                     ARCHIVO_MATRIZ);
27                 fclose(archivo);
28                 return EXIT_FAILURE;
29             }
30         }
31         fprintf(archivo, "\n");
32     }
33     fclose(archivo);
34     printf("[PROGRAMA 1] El archivo '%s' se ha generado exitosamente.\n",
35         ARCHIVO_MATRIZ);
36     return EXIT_SUCCESS;
37 }
```

Código 1.4. Programa 1: creación del archivo matriz.txt (programa1.c).

1.8.2 Código de programa2.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
```

```
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 #define FILAS 3
8 #define COLUMNAS 9
9 #define ARCHIVO_ORIGEN "matriz.txt"
10 #define ARCHIVO_DESTINO "resultados.txt"
11
12 void tarea_hijo(int col) {
13     FILE *f_in = fopen(ARCHIVO_ORIGEN, "r");
14     if (f_in == NULL) {
15         perror("[HIJO] Error al abrir archivo origen");
16         exit(EXIT_FAILURE);
17     }
18
19     int matriz[FILAS][COLUMNAS];
20     for (int i = 0; i < FILAS; i++) {
21         for (int j = 0; j < COLUMNAS; j++) {
22             if (fscanf(f_in, "%d", &matriz[i][j]) != 1) {
23                 fprintf(stderr, "[HIJO %d] Error al leer la matriz\n", col + 1);
24                 fclose(f_in);
25                 exit(EXIT_FAILURE);
26             }
27         }
28     }
29     fclose(f_in);
30
31     long long producto = 1;
32     for (int i = 0; i < FILAS; i++) {
33         producto *= matriz[i][col];
34     }
35
36     FILE *f_out = fopen(ARCHIVO_DESTINO, "a");
37     if (f_out == NULL) {
38         perror("[HIJO] Error al abrir archivo de resultados");
39         exit(EXIT_FAILURE);
40     }
41
42     if (fprintf(f_out, "Columna %d: %lld\n", col + 1, producto) < 0) {
43         fprintf(stderr, "[HIJO %d] Error al escribir resultado\n", col + 1);
44         fclose(f_out);
45         exit(EXIT_FAILURE);
46     }
47
48     fclose(f_out);
49     exit(EXIT_SUCCESS);
50 }
```

```
50 }
51
52 int main(void) {
53     FILE *f_check = fopen(ARCHIVO_ORIGEN, "r");
54     if (f_check == NULL) {
55         fprintf(stderr, "[PADRE] Error: No existe '%s'. Ejecute primero el
56             Programa 1.\n", ARCHIVO_ORIGEN);
57         return EXIT_FAILURE;
58     }
59     fclose(f_check);
60
61     FILE *f_init = fopen(ARCHIVO_DESTINO, "w");
62     if (f_init == NULL) {
63         perror("[PADRE] Error al inicializar el archivo de resultados");
64         return EXIT_FAILURE;
65     }
66     fclose(f_init);
67
68     printf("[PADRE] Creando %d procesos hijos para procesar las columnas en
69         paralelo...\n", COLUMNAS);
70     fflush(stdout); // Vacía el bufer antes de fork(): evita que cada hijo
71         repita este mensaje al terminar
72
73     for (int col = 0; col < COLUMNAS; col++) {
74         pid_t pid = fork();
75
76         if (pid < 0) {
77             perror("[PADRE] Error al ejecutar fork()");
78             return EXIT_FAILURE;
79         } else if (pid == 0) {
80             // Proceso Hijo
81             tarea_hijo(col);
82         }
83     }
84
85     for (int i = 0; i < COLUMNAS; i++) {
86         int status;
87         pid_t pid_hijo = wait(&status);
88         if (pid_hijo > 0) {
89             if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
90                 fprintf(stderr, "[PADRE] Advertencia: Proceso hijo (PID %d)
91                     termino con error.\n", pid_hijo);
92             }
93         }
94     }
95 }
```



```
92     printf("\n[PADRE] Todos los hijos han terminado. Resultados contenidos en
93         ' %s':\n", ARCHIVO_DESTINO);
94     printf("-----\n");
95     FILE *f_res = fopen(ARCHIVO_DESTINO, "r");
96     if (f_res == NULL) {
97         perror("[PADRE] Error al abrir el archivo de resultados");
98         return EXIT_FAILURE;
99     }
100
101     char linea[256];
102     while (fgets(linea, sizeof(linea), f_res) != NULL) {
103         printf(" %s", linea);
104     }
105     fclose(f_res);
106
107     printf("-----\n");
108     return EXIT_SUCCESS;
109 }
```

Código 1.5. Programa 2: proceso padre y procesos hijos (programa2.c).

1.8.3 Código del Makefile

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -std=c11
3
4 TARGET1 = program1
5 TARGET2 = programa2
6
7 all: $(TARGET1) $(TARGET2)
8
9 $(TARGET1): program1.c
10     $(CC) $(CFLAGS) -o $(TARGET1) program1.c
11
12 $(TARGET2): programa2.c
13     $(CC) $(CFLAGS) -o $(TARGET2) programa2.c
14
15 clean:
16     rm -f $(TARGET1) $(TARGET2) matriz.txt resultados.txt
17
18 .PHONY: all clean
```

Código 1.6. Makefile para compilar los dos programas.

Referencias

- [1] F. Márquez, *Unix: Programación Avanzada*. Grupo Editorial Ra-Ma, 2015.
- [2] «C Multidimensional Arrays,» W3Schools, visitado 28 de sep. de 2026. dirección: https://www.w3schools.com/c/c_arrays_multi.php
- [3] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ: Wiley, 2018.
- [4] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.
- [5] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.^a ed. Boston, MA: Pearson, 2014.
- [6] W. Stallings, *Operating Systems: Internals and Design Principles*, 9.^a ed. Londres, Reino Unido: Pearson, 2017.